

Débuter en Python

Variables, listes, conditions, boucles et fonctions
à partir de situations simples de chimie

OBJECTIF

Savoir lire, exécuter et modifier un petit programme. Cette fiche ne demande aucune bibliothèque : les huit scripts utilisent uniquement les outils de base de Python.

Principe de travail : exécuter une première fois le script sans le modifier, repérer les données, changer une seule valeur, puis prévoir le résultat avant de relancer.

PARCOURS CONSEILLÉ

Étape	Script	Ce que l'on apprend
1	01_variables_et_calculs.py	Nommer des données, calculer, afficher
2	02_conditions.py	Faire choisir une action au programme
3	03_boucle_for.py	Répéter une action
4	04_listes.py	Rassembler et modifier des mesures
5	05_moyenne_sans_bibliotheque.py	Construire un calcul pas à pas
6	06_fonction_simple.py	Réutiliser un calcul
7	07_minimum_maximum.py	Extraire des valeurs remarquables
8	08_tableau_de_mesures.py	Associer deux séries de données

Les nombres placés dans les scripts sont des exemples. On peut les remplacer par des données de TP, en conservant leurs unités.

1. Variables, calculs et affichage

Une variable

Une variable associe un nom à une valeur. Un nom explicite rend le programme plus facile à relire.

```
concentration = 0.100
volume_ml = 12.4
volume_litre = volume_ml / 1000
```

Les opérations essentielles

Python utilise +, -, * et /. La puissance s'écrit **. Les parenthèses précisent l'ordre des opérations.

```
quantite = concentration * volume_litre
carre = volume_ml ** 2
print("Quantité :", quantite, "mol")
```

À essayer

Dans le script 01, remplacer 12,4 mL par 15,0 mL. Prévoir si la quantité de matière va augmenter ou diminuer, puis vérifier.

2. Conditions : faire un choix

Le test

Une condition produit une réponse vraie ou fausse. Le bloc indenté sous **if**, **elif** ou **else** est exécuté selon cette réponse.

```
if ph < 7:
    print("Solution acide")
elif ph == 7:
    print("Solution neutre")
else:
    print("Solution basique")
```

Attention : = attribue une valeur ; == compare deux valeurs. L'indentation de quatre espaces indique les instructions appartenant à la condition.

À essayer

Exécuter le script 02 avec pH = 7, puis pH = 2,5. Ajouter ensuite un message spécial lorsque le pH est inférieur à 2.

3. Listes et boucles

La liste

Une liste conserve plusieurs valeurs dans un ordre précis. La première valeur porte l'indice 0.

```
volumes = [12.1, 12.3, 12.2, 12.4]
print(volumes[0])
print(len(volumes))
```

La boucle for

La boucle prend successivement chaque valeur de la liste. Le nom choisi après **for** désigne la valeur en cours.

```
somme = 0
for volume in volumes:
    somme = somme + volume
moyenne = somme / len(volumes)
```

Lire le bloc de gauche à droite : la somme vaut d'abord 0 ; chaque volume lui est ajouté ; la division n'est faite qu'après la boucle.

4. Fonctions

Pourquoi une fonction ?

Une fonction donne un nom à un calcul et permet de l'utiliser plusieurs fois sans le recopier.

```
def quantite_matiere(concentration, volume_ml):
    volume_litre = volume_ml / 1000
    quantite = concentration * volume_litre
    return quantite

n = quantite_matiere(0.100, 12.5)
```

def commence la définition. Les paramètres sont les données reçues. **return** renvoie le résultat calculé.

Petits défis

1. Ajouter une cinquième mesure dans le script 05. 2. Transformer le script 07 pour traiter des masses. 3. Dans le script 06, calculer trois quantités de matière avec la même fonction.

5. Choisir le bon script

Besoin	Script de départ	Exemples de TP
Faire un calcul avec unités	01 ou 06	Dosages, dilutions, rendements
Prendre une décision	02	Acide/base, critère de comparaison
Parcourir des mesures	03 ou 04	Tous les TP quantitatifs
Calculer une moyenne	05	TP1, TP6, TP8, TP9, TP13, TP14
Trouver des extrêmes	07	Température, signal, rendement
Associer x et y	08	Titrages, cinétique, potentiométrie

Méthode de débannage

1. Lire la dernière ligne du message d'erreur. 2. Repérer le numéro de ligne indiqué. 3. Vérifier les parenthèses, les deux-points et l'indentation. 4. Vérifier que chaque nom de variable est écrit exactement de la même façon.

Erreurs fréquentes

- Utiliser une virgule décimale : Python attend **12.5**, pas **12,5**.
- Oublier les deux-points après **if**, **else**, **for** ou **def**.
- Mélanger les unités, par exemple mL et L.
- Modifier plusieurs lignes à la fois sans tester entre les modifications.

Avant de conclure

Un programme qui s'exécute peut encore produire un résultat faux. Toujours vérifier l'unité, l'ordre de grandeur et le sens physique du résultat.

Validation rapide

Je sais : identifier les données d'un script ; modifier une valeur ; expliquer le rôle d'une boucle ; ajouter une valeur à une liste ; écrire une condition simple ; appeler une fonction ; contrôler la vraisemblance du résultat.